

Reviewer Pack — Hypergeometric Function Rank of Boolean Functions Bounds Com...

Entry 7b2b667cc100 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 11:46:28 UTC

Hypergeometric Function Rank of Boolean Functions Bounds Communication Complexity for XOR

Entry ID: 7b2b667cc100 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 11:46:28 UTC

1. Conjecture

Field A (mathematical branch): Hypergeometric Functions and Special Functions **Field B** (complexity object): Communication Complexity

Statement:

[‘For any boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the hypergeometric function rank r_f of f is defined as the smallest integer such that the polynomial representation of f has a non-zero coefficient for each monomial with degree $\leq r_f$. Then, for all $n \geq 3$, the communication complexity of XOR on n bits, $CC_XOR(n)$, satisfies $CC_XOR(n) = \Omega(r_f \log n)$.’, ‘For any boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, if there exists an input x such that $f(x) \neq 0$, then the hypergeometric function rank r_f of f is at least 2.’, ‘If $CC_XOR(n) < \Omega(r_f \log n)$, then there exists a boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$ with hypergeometric function rank $r_f < \log n$.’]

Rationale (proposer’s reasoning):

[‘Hypergeometric functions are a well-studied class of special functions that have been applied to various areas in mathematics and computer science. Their ranks can provide insights into the complexity of certain problems. By linking hypergeometric function ranks with communication complexity, this conjecture aims to explore new connections between mathematical theory and practical computational tasks.’, ‘Previous work has shown that certain properties of boolean functions, such as their Fourier transform coefficients or their representation as polynomials, are related to communication complexity. This conjecture builds on these ideas by considering the hypergeometric function rank as a potential indicator of communication complexity.’, ‘The use of hypergeometric functions in this context is novel and could potentially expose new structures that are not apparent when using more traditional measures of boolean function complexity.’]

Taxonomy category: HypergeometricFunctionRank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8d733c7beeb59620

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of randomly generated boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, with n in $[3, 40]$, and across 30 seeds, the inequality $CC_XOR(n) = \Omega(r_f \log n)$ holds true with a mean metric_mean ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "hypergeometric function rank" AND "communication complexity" AND XOR - "Boolean function" AND "hypergeometric function rank" AND $CC_XOR(n)$ - "communication complexity" $\leq \Omega$ ("hypergeometric function rank" $\log n$)

Top relevant hits considered: - [<http://arxiv.org/abs/2510.17487v1>] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [<http://arxiv.org/abs/1411.4413v2>] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [<http://arxiv.org/abs/2601.07595v3>] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def hypergeometric_rank(f):
        n = int(math.log2(len(f)))
        if n == 0:
            return 0
        A = [[0] * (n + 1) for _ in range(n + 1)]
        b = [0] * (n + 1)
        for i in range(1, n + 1):
            for j in range(i + 1):
                A[i][j] = math.comb(i, j)
```

```

        b[i] = sum(f[j] for j in range(2**i))

# Gaussian elimination
for i in range(n + 1):
    if A[i][i] == 0:
        return float('inf')
    for j in range(i + 1, n + 1):
        factor = A[j][i] / A[i][i]
        for k in range(i, n + 1):
            A[j][k] -= factor * A[i][k]
        b[j] -= factor * b[i]

rank = 0
for i in range(n + 1):
    if any(A[i][j] != 0 for j in range(rank)):
        rank += 1

return rank

def communication_complexity(n):
    return n

instances_tested = 30
metric_values = []

for _ in range(instances_tested):
    f = generate_boolean_function(random.randint(3, 40))
    r_f = hypergeometric_rank(f)
    if r_f == float('inf'):
        continue
    CC_XOR_n = communication_complexity(len(f))
    metric_values.append(CC_XOR_n / (r_f * math.log2(len(f))))

if not metric_values:
    return {
        "metric_name": "CC_XOR(n) / (r_f * log n)",
        "metric_value": 0,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_metric = sum(metric_values) / len(metric_values)
std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
conjecture_holds = all(x >= 1 for x in metric_values)
counterexample = "" if conjecture_holds else "CC_XOR(n) <  $\Omega(r_f \log n)$ "

return {
    "metric_name": "CC_XOR(n) / (r_f * log n)",
    "metric_value": mean_metric,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":

```

```

import sys
seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric = sum(x["metric_value"] for x in results) / len(results)
std_metric = math.sqrt(sum((x["metric_value"] - mean_metric) ** 2 for x in results) / len(results))
support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

if all(x["conjecture_holds"] for x in results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
else:
    first_failing_seed = next(x["seed"] for x in results if not x["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"CC_XOR(n) < Ω(r_f log n)\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the expected time frame. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time constraints.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13772
2	preregistration	ollama_remote	glm4:latest	0	0	6163
3	novelty	ollama_remote	glm4:latest	0	0	4708
4	novelty	ollama_remote	glm4:latest	0	0	10540
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15732

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6132
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14065
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11414
9	judge	ollama_remote	glm4:latest	0	0	16073

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98600 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7b2b667cc100.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7b2b667cc100.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7b2b667cc100.tar.gz (if generated)